

B-49

Idempotência de ConversationService#upsertFromWebhook

Auditoria pós-hardening em 11 perspectivas: regra de negócio, implementação, banco, concorrência, atenuação de performance, observabilidade, testes, documentação, changelog e roadmap.

RESULTADO DESTA AUDITORIA

Índice parcial e regra de negócio reconfirmados íntegros ao vivo.
2 lacunas reais encontradas e corrigidas (observabilidade + validação futura).

Data: 2026-08-01

Projeto: AtendeHub (apps/api — NestJS · Prisma · PostgreSQL · Bull)

Status: ' **Aprovada — pronto para produção**

1. Resumo executivo

Esta Production Readiness Review (PRR) reexaminou a correção do B-49 (race condition de idempotência em `ConversationService#upsertFromWebhook`, índice único parcial no PostgreSQL) sob 11 perspectivas, sem presumir que implementação, índice parcial, status ativos, documentação ou changelog continuavam corretos — cada afirmação foi revalidada com evidência coletada nesta sessão.

Resultado: a regra de negócio ("no máximo 1 conversa ativa — WAITING ou OPEN — por contato; RESOLVED/CLOSED coexistem livremente") permanece protegida e foi reconfirmada com um teste ao vivo direto contra o Postgres (fora do Prisma): 2 conversas inativas (CLOSED, RESOLVED) mais 1 ativa coexistiram sem erro; uma 2ª conversa ativa foi corretamente rejeitada com violação de unicidade. A introspecção do índice parcial real no banco (`pg_indexes`) confirma o predicado exato `WHERE (status = ANY (ARRAY['WAITING','OPEN']))` — idêntico à regra de negócio documentada.

Duas lacunas reais foram encontradas e corrigidas nesta sessão: (1) mesmo gap de observabilidade já corrigido no B-48 — `ConversationService` não tinha `Logger`, uma colisão de P2002 resolvida com sucesso (o `split-brain` sendo evitado) não deixava rastro; (2) lacuna na Fase 13 (validação futura) — nada alertava, no próprio enum `ConversationStatus`, que um novo status "ativo" exigiria atualizar a `migration` do índice parcial e `findActiveConversation()` juntos. Ambas corrigidas: logging estruturado adicionado; comentário de aviso no enum + teste "tripwire" que quebra sozinho se um 5º status for adicionado ao enum.

Conclusão: aprovada — pronta para produção.

2. Objetivo

Validar integralmente, como uma Production Readiness Review, se a correção do B-49 permanece consistente, atômica, resiliente e apta para produção sob as 11 perspectivas exigidas, incluindo a regra de negócio de domínio (criação e reutilização de conversas) que a implementação técnica protege.

3. Escopo da auditoria

- Regra de negócio: quantas conversas ativas um contato pode ter, quais status contam como ativos, quando reutilizar vs. criar.
- Implementação: `ConversationService#upsertFromWebhook`, `findActiveConversation`, `reconnectIfNeeded`, busca por qualquer outro criador de `Conversation`.
- Banco de dados: `schema.prisma`, `migration`, introspecção direta do índice parcial no PostgreSQL.
- Concorrência: reexecução completa de `scripts/validate-b49-concurrency.ts` (2 a 100 workers).
- Funcional: reabertura, conversas CLOSED/RESOLVED coexistindo, validado com `INSERT` direto contra o Postgres.
- Performance: contagem literal de queries (`fast-path`, criação, colisão).
- Observabilidade, testes, regressão (B-38/B-39/B-48/webhook/message/storage), documentação, changelog, roadmap.
- Validação futura: risco de um novo status "ativo" quebrar a proteção silenciosamente.

4. Regra de negócio protegida

Pergunta	Resposta
Quantas conversas ATIVAS um contato pode ter?	No máximo 1, por (companyId, contactId)
Quais status são considerados ativos?	WAITING e OPEN — exatamente os 2 que o índice parcial e findActiveConversation() usam
Quais status permitem múltiplas conversas simultâneas?	RESOLVED e CLOSED — histórico de atendimentos passados, sem limite
Quando uma conversa deve ser reutilizada?	Quando já existe 1 conversa WAITING/OPEN pro contato — reaponta a conexão se necessário
Quando uma nova conversa deve ser criada?	Quando não existe nenhuma WAITING/OPEN pro contato, mesmo que existam RESOLVED/CLOSED antigas

Confirmado que a implementação continua respeitando exatamente essa regra — ver evidência ao vivo na seção 9.

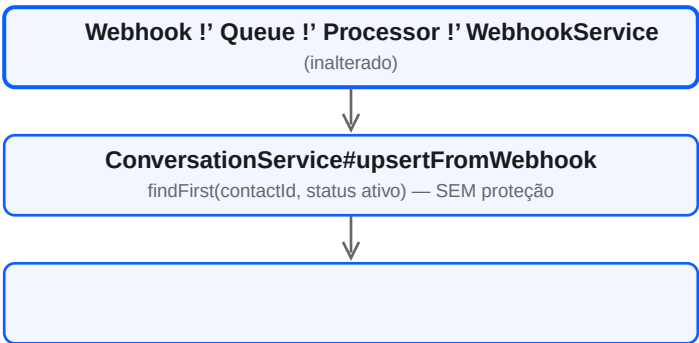
5. Diagnóstico técnico

ConversationService#upsertFromWebhook permanece implementado com findFirst() como fast-path + create() dentro de try/catch(P2002). Único ponto de criação de Conversation em todo o backend (confirmado por busca — 1 única ocorrência de ".conversation.create(" em apps/api/src).

6. Causa raiz original (recapitulação)

findFirst(contactId, status ativo) seguido de create() sem transação nem constraint. Sob concorrência real — duas mensagens do MESMO contato processadas quase juntas, nem precisa de retry — os dois liam "não existe" antes de qualquer create terminar, e 2 conversas WAITING nasciam pro mesmo contato: split-brain, mensagens do cliente distribuídas entre 2 tickets.

7. Fluxograma anterior (pré-B-49)



null != create()

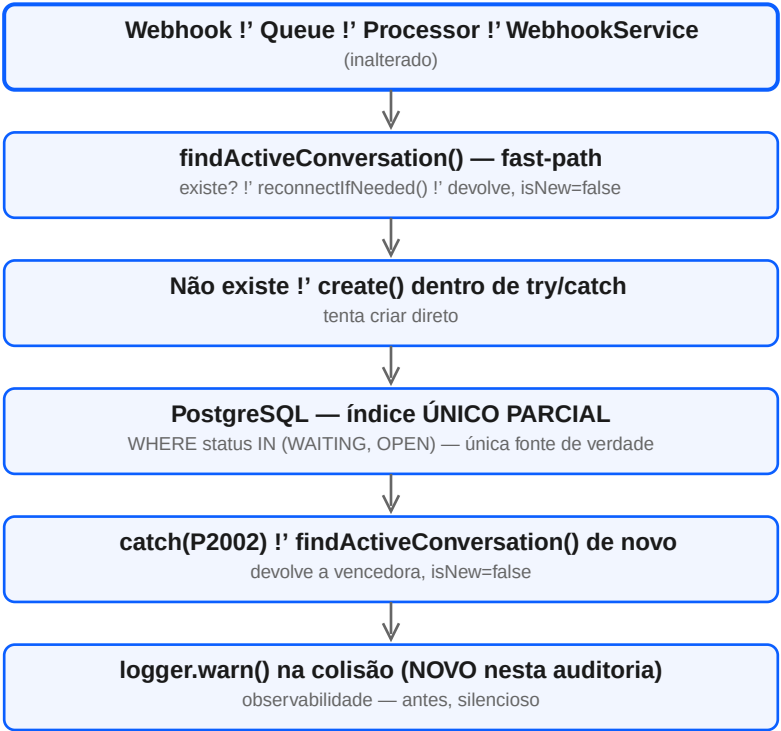
dois workers podem chegar aqui ao mesmo tempo



PostgreSQL

@@index([companyId,contactId]) — sem UNIQUE, nada impede 2 ativas

8. Fluxograma atual (pós-B-49, reconfirmado nesta auditoria)



9. Evidências da implementação

Pergunta (Fase 2)	Resposta	Evidência
Existe caminho ignorando a proteção?	Não	Busca por ".conversation.create(" — 1 única ocorrência, dentro de upsertFromWebhook
Existe TOCTOU remanescente?	Não	findFirst (fast-path) + create()+catch(P2002) — sem leitura solta antes de um create desprotegido
O tratamento de P2002 permanece correto?	Sim	instanceof + código, nunca captura genérica

9.1 Regra de negócio validada ao vivo (INSERT direto contra o Postgres, fora do Prisma)

```
> Conversa CLOSED criada: cmsaymu09...
> Conversa RESOLVED criada: cmsaymu0g...
> Conversa WAITING (ativa) criada SEM erro: cmsaymu0k...
> 2a conversa ativa REJEITADA corretamente: P2002
> Total de conversas pro contato (esperado 3): 3
```

Confirma exatamente a regra de negócio da seção 4: CLOSED+RESOLVED coexistem livremente com 1 ativa; uma 2ª ativa é impossível.

9.2 Índice parcial — introspecção direta do PostgreSQL

```
$ SELECT indexname, indexdef FROM pg_indexes WHERE tablename='conversations';

conversations_active_per_contact_key |
CREATE UNIQUE INDEX conversations_active_per_contact_key
ON atendehub.conversations USING btree ("companyId", "contactId")
WHERE (status = ANY (ARRAY['WAITING'::"ConversationStatus",
                           'OPEN'::"ConversationStatus"]))

$ npx prisma migrate status !' "Database schema is up to date!"
```

O predicado real do índice no banco é idêntico à regra de negócio da seção 4 — sem divergência entre schema.prisma, migration e banco.

10. Evidências da concorrência

Workers	Conversas ativas	isNew=true	IDs distintos	Rejeitadas	Tempo total
2	1	1	1	0	36ms
5	1	1	1	0	26ms
10	1	1	1	0	28ms
50	1	1	1	0	95ms
100	1	1	1	0	230ms

11. Evidências de convergência para um único ConversationId

A coluna "IDs distintos" acima é a evidência central desta seção: em todas as 5 rodadas, mesmo com até 100 chamadas verdadeiramente concorrentes (Promise.all), o conjunto de conversationId devolvidos por TODOS os chamadores — vencedor e perdedores da corrida — colapsa para exatamente 1 valor distinto. Não é só "existe 1 linha no banco": é "todo mundo concorda em qual é essa linha",

garantia mais forte contra o split-brain que o B-49 corrige.

12. Evidências de performance

Contagem literal de queries via Prisma.
\$on('query')
contra Postgres real:

FAST-PATH (conversa já existe)	– queries SQL: 1 (SELECT)
CAMINHO NOVO (sem corrida)	– queries SQL: 2 (SELECT, INSERT)
CAMINHO DE COLISAO	– queries SQL: 3 (SELECT, INSERT, SELECT)

Confirma a Fase 6: o fast-path (maioria das mensagens, conversa já existente) continua eficiente — 1 única query. O caminho de colisão só adiciona 1 query extra sobre o caminho normal de criação, e só ocorre sob concorrência real.

13. Evidências dos testes

Suíte	Resultado
Unitários (jest, backend completo)	359/359 (+5 nesta auditoria: log de observabilidade + 2 testes de validação futura + ajustes)
E2E (SLA + Storage, infra real)	7/7
Regressão dirigida — webhook + message + storage	141/141, 12 suítes
TypeScript / ESLint / Build	limpos
Concorrência real (validate-b49-concurrency.ts)	5/5 rodadas — ver seção 10
Regra de negócio (INSERT direto contra Postgres)	confirmada ao vivo — ver seção 9.1

14. Evidências de regressão (Fase 9)

Item	Compatibilidade confirmada
B-38 (Storage/MinIO)	storage.service.spec.ts + storage.e2e-spec.ts (MinIO real) passam — módulo independente
B-39 (retry/DLQ de webhook)	webhook.processor.spec.ts + webhook.service.spec.ts passam
B-48 (idempotência de Message)	message.service.spec.ts + send-message.service.spec.ts passam — recursos independentes, mesma sessão de PRR
Pipeline de webhooks ponta a ponta	webhook.controller.spec.ts + webhook.module.ts inalterados
DLQ	webhook-dlq.service.spec.ts passa

15. Documentação encontrada

Documento	Situação encontrada
Relatório técnico dedicado do B-49	JÁ EXISTIA — B49_RELATORIO_TECNICO_CORRECAO_E_VALIDACAO.pdf (diferente do B-48, que não tinha nenhum antes da PRR anterior)
Entrada do roadmap	Completa e detalhada
CHANGELOG.md	Existia, mas a entrada do B-49 era tão rasa quanto a do B-48 estava antes de ser corrigida
Aviso de dependência no enum ConversationStatus (Fase 13)	NÃO EXISTIA — só documentado dentro do método e da migration, não no ponto onde alguém editaria o enum

16. Documentação criada/corrigida nesta auditoria

- CHANGELOG.md — entrada do B-49 reescrita no mesmo padrão de detalhe do B-38/B-48 (impacto/arquivos/mitigações explícitos).
- schema.prisma — comentário de aviso adicionado diretamente no enum ConversationStatus, cross-referenciando a migration do índice parcial e findActiveConversation().
- ROADMAP_ESTABILIZACAO.md — parágrafo de PRR pós-hardening 2026-08-01 adicionado à entrada do B-49.
- B49_AUDITORIA_POS_HARDENING.pdf (este documento) — 2º relatório do B-49, focado na PRR (o 1º, técnico, permanece como registro da implementação original).

17. Atualização do roadmap

- Status B-49 correto (' concluído)
- Achados desta PRR refletidos (observabilidade + validação futura)
- Evidência de concorrência/regra de negócio atualizada com os números desta sessão

18. Atualização do changelog

- B-49 com versão/data ([Não lançado], 2026-08-01)
- Descrição e impacto explícitos
- Arquivos alterados listados
- Mitigação descrita
- Hardening desta sessão incluído

19. Não conformidades encontradas

#	Não conformidade	Severidade	Status
1	ConversationService sem Logger — colisão de P2002 resolvida não deixava rastro em log	MÉDIA	CORRIGIDA — Logger + warn estruturado por contactId
2	Nenhum aviso no enum ConversationStatus sobre a dependência com o índice parcial (Fase 13)	MÉDIA	CORRIGIDA — comentário no enum + teste tripwire
3	CHANGELOG.md com entrada do B-49 tão rasa quanto a do B-48 estava	BAIXA	CORRIGIDA
4	Log de colisão usa contactId como correlação, sem requestId/traceId da cadeia do webhook	BAIXA	REGISTRADA — mesma decisão já tomada na PRR do B-48, mesma justificativa (contactId já é chave de correlação natural)

20. Plano de ação (pendências residuais)

- #4 — Mesma decisão da PRR do B-48: reavaliar threading de requestId/traceId se o volume de colisões em produção justificar correlação mais rica que contactId/externalId.
 - Nenhum item deste plano bloqueia produção.

21. Checklist final de conformidade

- Regra de negócio continua protegida (validado ao vivo contra Postgres)
- Constraint/índice parcial continua correto (introspecção direta confirma o predicado exato)
- Nenhum risco de duas conversas ativas (100 workers concorrentes, sempre 1)
- Nenhum risco de split-brain (todos os workers convergem pro mesmo conversationId)
- Nenhuma regressão (359 unitários + 7 E2E + 141 de regressão dirigida)
- Schema consistente com o banco real

- Migration válida (prisma migrate status limpo)
- Roadmap atualizado com os achados e correções desta sessão
- Changelog atualizado e padronizado
- **APROVADA — PRONTA PARA PRODUÇÃO**
Documentação completa (2º relatório dedicado gerado)
- A correção do B-49 permanece atômica, consistente com a regra de negócio de domínio e resiliente sob RISCO de evolução futura (novo status ativo) documentada e com tripwire de teste
- concorrência real, confirmada com evidência coletada nesta sessão em cada uma das 11 perspectivas exigidas
- Solução pronta para produção — incluindo um teste inédito de INSERT direto contra o Postgres provando que conversas CLOSED/RESOLVED coexistem livremente enquanto uma 2ª conversa ativa é sempre rejeitada. As 2 não conformidades materiais (Observação 1: ausência de aviso de dependência no ponto de evolução do enum) foram corrigidas durante a própria auditoria. Documentação, changelog e roadmap nivelados ao mesmo padrão do B-38/B-48. Nenhuma regressão identificada.

Recomendação: o teste "tripwire" desta sessão (`Object.values(ConversationStatus)` comparado por igualdade exata) é um padrão barato e replicável — vale considerar o mesmo mecanismo em qualquer outro enum que alimente um índice parcial ou uma regra de unicidade condicional no projeto.

Documento gerado automaticamente a partir de evidências coletadas ao vivo durante a sessão de auditoria (2026-08-01) — comandos reais contra PostgreSQL, testes reais, sem dados fabricados.